



PYTHON BOOTCAMP

www.jomhack.com

AI AGENTS



Core Functions

- **Reason:** Thoughts about what to do
- **Action:** Interact with external tools
- Observe results from each actions
- Continue cycle until completing a task

Use Cases

- **Q&A:** Using search tool for current information
- **Automation:** Break down complex task into steps
- **Data Analysis:** Querying databases and processing results

R.A.G



Retrieval:

- Searches and fetches relevant information from external knowledge sources.

Augmented:

- Enriches the original prompt with retrieved information.

Generation:

- Produces the final answer using both the query and augmented context.

BACKEND



Libraries:

- pip install fastapi uvicorn python-dotenv pymongo langchain langchain-google-genai

API:

- MONGODB_ATLAS_CLUSTER_URI
- GOOGLE_API_KEY

Resources:

- <https://www.mongodb.com/cloud/atlas>
- <https://aistudio.google.com/app/apikey>

BACKEND



.env:

- GOOGLE_API_KEY=
- GEMINI_MODEL=gemini-2.0-flash
- MONGODB_ATLAS_CLUSTER_URI=

BACKEND



agents/chat_agent.py:

```
backend > agents > chat_agent.py > ...
1  import os
2  import sys
3
4  sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
5
6  from langchain.agents import create_agent
7  from utils import llm, tools
8
9  SYSTEM_PROMPT = """
10  ## PERSONA
11  You are a poetic AI assistant that always responds in rhymes.
12
13  ## TASK
14  Your task is to assist users with their inquiries while adhering to your poetic nature.
15  Answer general questions directly without using tools.
16  ONLY use tools when the user explicitly asks about stored/personal data or posts.
17
18  ## TOOL CALLING
19  **IMPORTANT**: Only use tools when the user asks about personal posts, database content, or stored articles.
20  For general questions, greetings, or casual conversation, answer directly WITHOUT using any tools.
21
22  ## GUARDRAIL
23  Always respond in rhymes, no matter the question or task.
24  Politely refuse to answer anything related to violence, hate speech, or illegal activities.
25  """
```

BACKEND



agents/chat_agent.py:

```
27 def create_chat_agent():
28     """Factory function to create a new chat agent with its own memory."""
29     agent = create_agent(
30         model=llm,
31         tools=tools,
32         system_prompt=SYSTEM_PROMPT,
33     )
34
35     return agent
36
37 # Uncomment below to test the agent directly
38
39 if __name__ == "__main__":
40     test_agent = create_chat_agent()
41
42     # Test general question (should NOT use database tool)
43     print("\n=== Testing general question ===")
44     response = test_agent.invoke({
45         "messages": [{"role": "user", "content": "Tell me about Malaysia."}]
46     })
47     print("Response:", response['messages'][-1].content)
48
49     # # Test database question (should use database tool)
50     # print("\n=== Testing database question ===")
51     # response = test_agent.invoke({
52     #     "messages": [{"role": "user", "content": "Tell me about my personal posts"}]
53     # })
54     # print("Response:", response['messages'][-1].content)
```

BACKEND



agents/__init__.py:

```
backend > agents >  __init__.py > ...  
1  from .chat_agent import create_chat_agent  
2  
3  __all__ = ["create_chat_agent"]
```


BACKEND



utils/llm.py:

```
backend > utils > llm.py > ...
1  import os
2  from dotenv import load_dotenv
3  from langchain_google_genai import ChatGoogleGenerativeAI
4
5  load_dotenv()
6
7  api_key = os.getenv("GOOGLE_API_KEY")
8  model = os.getenv("GEMINI_MODEL")
9
10 if not api_key:
11     raise ValueError("GOOGLE_API_KEY environment variable is not set")
12
13 llm = ChatGoogleGenerativeAI(
14     model=model,
15     api_key=api_key,
16     temperature=0.2,
17 )
```

BACKEND



utils/tools.py:

```
backend > utils > tools.py > ...
1  import os
2  from langchain_core.tools import tool
3  from pymongo import MongoClient
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  @tool
9  def post_data_from_db(query: str) -> str:
10     """
11     Fetch posts from MongoDB database.
12
13     ONLY use this tool when the user explicitly asks about:
14     - Your personal posts
15     - Posts from the database
16     - Content you have written or saved
17     - Blog posts or articles in your collection
18
19     DO NOT use this tool for:
20     - General knowledge questions
21     - Greetings or casual conversation
22     - Questions about topics not related to stored posts
23
24     Args:
25     |   query: The user's question about posts
26
27     Returns:
28     |   A formatted string of posts from the database or an error message
29     """
```

```
30     try:
31         client = MongoClient(os.getenv("MONGODB_ATLAS_CLUSTER_URI"))
32         db = client["example_db"]
33         collection = db["posts"]
34
35         # Fetch all posts (you can modify this to search by title, content, etc.)
36         posts = list(collection.find({}).limit(5)) # Limit to 5 posts for brevity
37
38         if posts:
39             # Format the posts nicely
40             result = []
41             for post in posts:
42                 post_info = f>Title: {post.get('title', 'N/A')}, Content: {post.get('content', 'N/A')}"
43                 result.append(post_info)
44             return "\n".join(result)
45         else:
46             return "No posts found in the database."
47     except Exception as e:
48         return f"Error fetching data: {str(e)}"
49     finally:
50         client.close()
51
52
53     tools = [post_data_from_db]
```

BACKEND



utils/__init__.py:

```
backend > utils >  __init__.py > ...  
1  from .llm import llm  
2  from .tools import tools  
3  
4  __all__ = [  
5      "llm",  
6      "tools"  
7  ]
```

BACKEND



api/main.py:

```
backend > api > main.py > ...
1  from fastapi import FastAPI, HTTPException
2  from fastapi.middleware.cors import CORSMiddleware
3  from pydantic import BaseModel, Field
4  from typing import List, Optional
5  import sys
6  import os
7
8  # Add parent directory to path to import agents
9  sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
10
11 from agents import create_chat_agent
12
13 app = FastAPI(title="Chat API")
14
15 # Configure CORS
16 app.add_middleware(
17     CORSMiddleware,
18     allow_origins=["http://localhost:5173"], # React Router dev server
19     allow_credentials=True,
20     allow_methods=["*"],
21     allow_headers=["*"],
22 )
```

```
24 class ChatMessage(BaseModel):
25     message: str = Field(..., description="User's message to the chatbot")
26
27 class ChatResponse(BaseModel):
28     response: str = Field(..., description="Bot's response")
29     error: Optional[str] = None
30
31 @app.get("/api/health")
32 async def health_check():
33     """Health check endpoint"""
34     return {"status": "ok"}
```

BACKEND



api/main.py:

```
37 @app.post("/api/chat", response_model=ChatResponse)
38 async def chat(chat_message: ChatMessage):
39     """
40     Chat endpoint that processes user messages through the AI agent.
41     """
42     try:
43         agent = create_chat_agent()
44         # Use messages format as expected by create_agent
45         result = agent.invoke({
46             "messages": [{"role": "user", "content": chat_message.message}]
47         })
48
49         # Extract the last message content from the response
50         last_message = result.get("messages", [])[-1] if result.get("messages") else None
51         response_text = last_message.content if last_message else "I couldn't generate a response."
52
53         return ChatResponse(
54             response=response_text,
55             error=None
56         )
57     except Exception as e:
58         raise HTTPException(
59             status_code=500,
60             detail=f"Error processing message: {str(e)}"
61         )
62
63 if __name__ == "__main__":
64     import uvicorn
65     uvicorn.run(app, host="0.0.0.0", port=8000)
```


BACKEND



Full Code:

- https://github.com/Bro-chill/Boilerplate_Collection/tree/main/chatbot_rag